

Actually, if **agent maneuverability** is your main priority, **Git submodules might actually slow your agent down and cause friction.**

While submodules sound great on paper for linking repositories, they lock specific commits in place. If your agent is autonomously writing code, updating its own logic, or expanding its local knowledge base, submodules require constant, manual commit pinning from the parent repo. If the agent updates `aesop-xi-protocol`, the parent repo won't see it until a manual update command is run. It creates a rigid bottleneck.

For an adaptive, agentic assistant operating within an Android environment, you have much faster and more flexible architectural pathways to achieve total maneuverability.

1. The Dynamic Pathway: Android Local IPC (System Services)

Since **Horizons UI** is an Android app, the most seamless approach is to treat your backend modules as local background services.

- **How it works:** **NovusÆxenti** and **Æsop-Xi** run as an Android Service or an isolated process. They expose an interface (using **AIDL** or local broadcast receivers) that **Horizons UI** connects to dynamically.
- **Why it's best for agents:** Your agent can pull data from **NovÆ-Corpus** or rewrite execution protocols in real time without needing a Git update to refresh the user interface. The UI simply displays whatever the running engine outputs.

2. The Monorepo Twist: A Unified "Workspace"

If you absolutely want all the code in a single place so your agent can see and modify everything at once, skip submodules and use a **Gradle multi-module project (or Monorepo)**.

- **How it works:** You have one single Git repository, but inside it, you split the code into completely separate, independent folders:

```
└─ horizons-workspace
├─ app-horizons-ui      (Android UI module)
├─ core-novaecopia      (Agent logic module)
├─ protocol-aesop-xi    (Execution harness module)
└─ data-nova-corpus     (Database / Wiki module)
```
- **Why it's best for agents:** Your agent doesn't have to deal with multiple Git authentication keys or detached heads. It can seamlessly read a file in `data-nova-corpus` and instantly use it to update code in `protocol-aesop-xi` within a single commit.

The Recommendation

- If you want the agent to have **absolute freedom to read, write, and re-architect the entire system across layers**, use the **Unified Monorepo Workspace**.
- If you want to keep the repositories completely isolated on GitHub but connected smoothly on the Android device itself, use **Android Local IPC / System Services**.

How autonomous do you want this agent to be? Will it actively be **writing and patching its own code** within these repos, or is it mostly navigating and reading the data inside them?